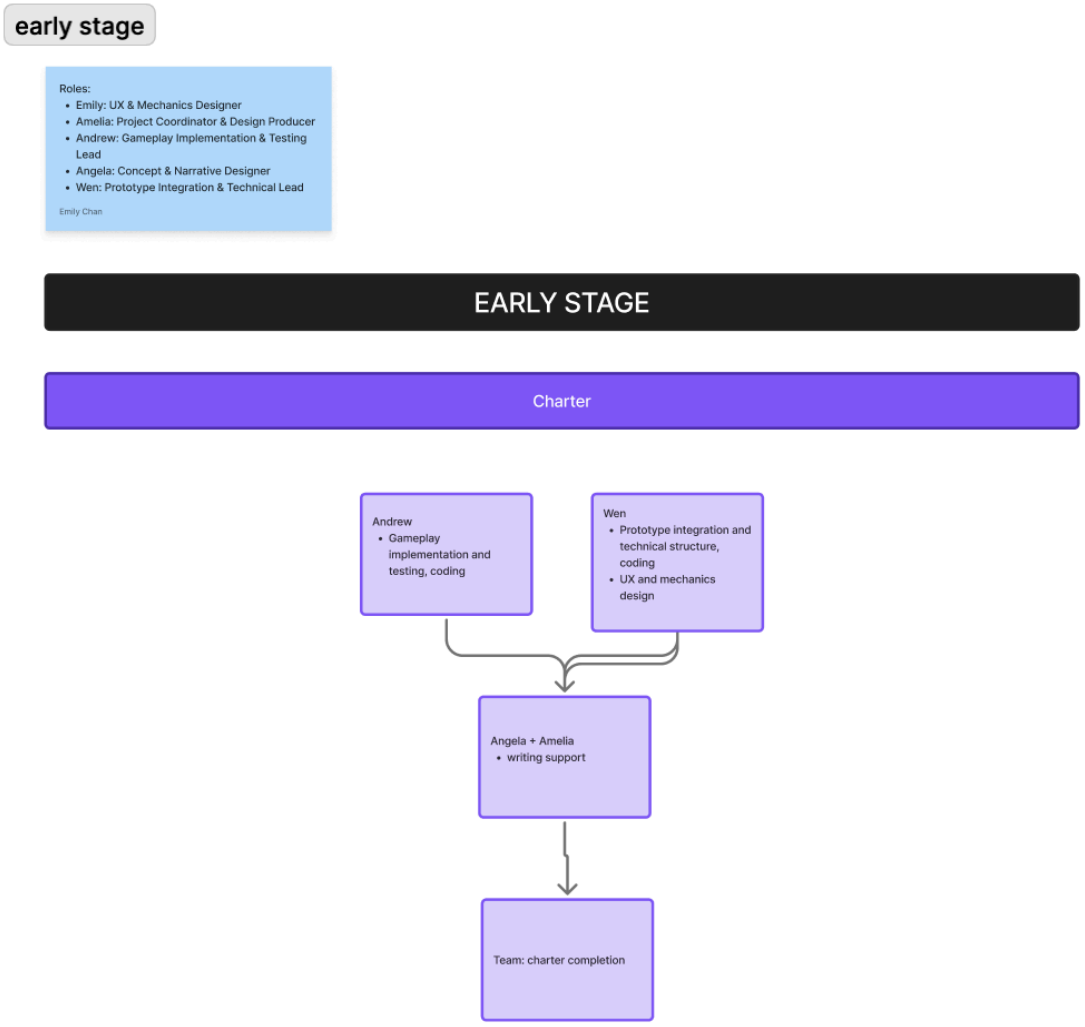
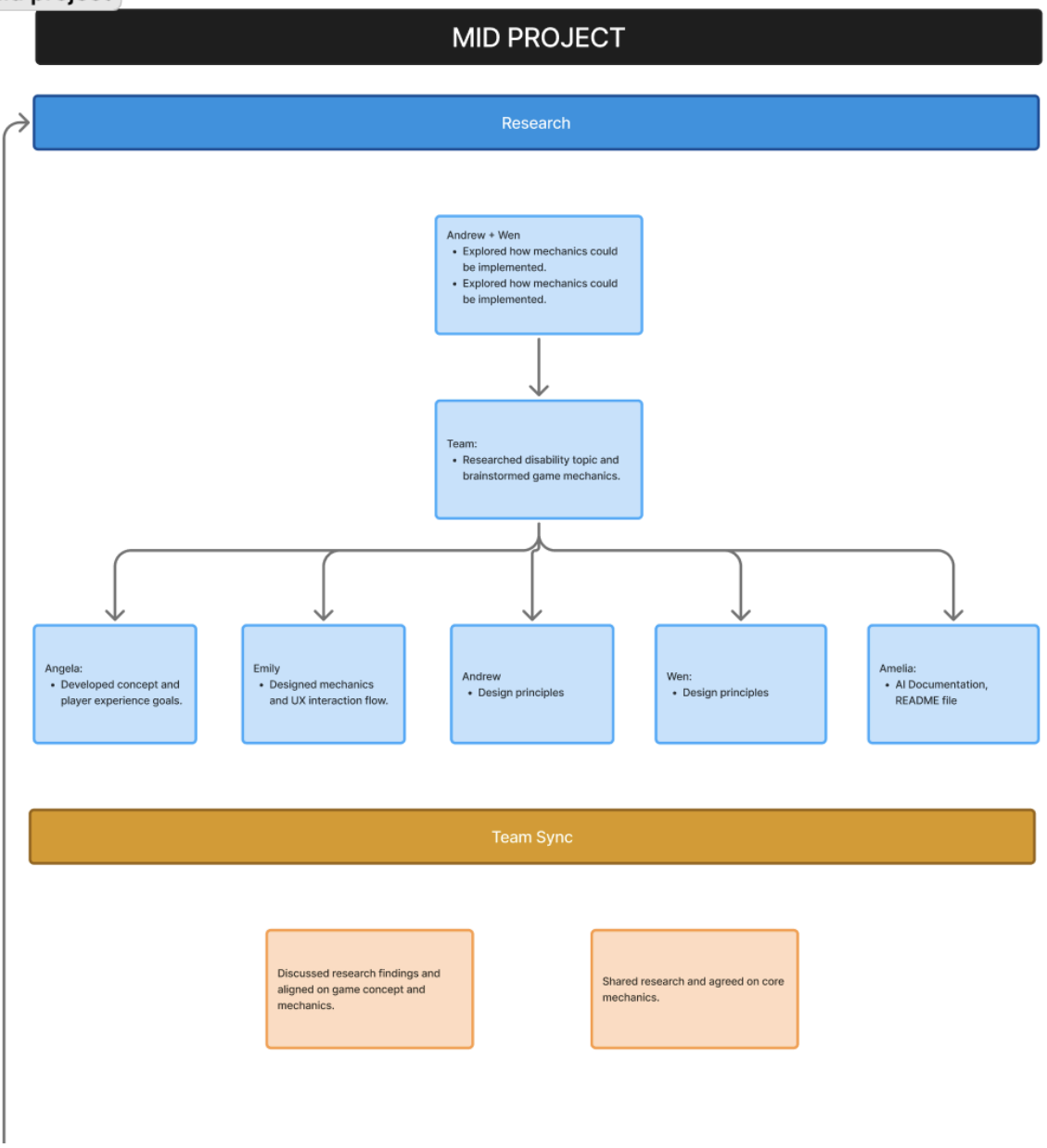
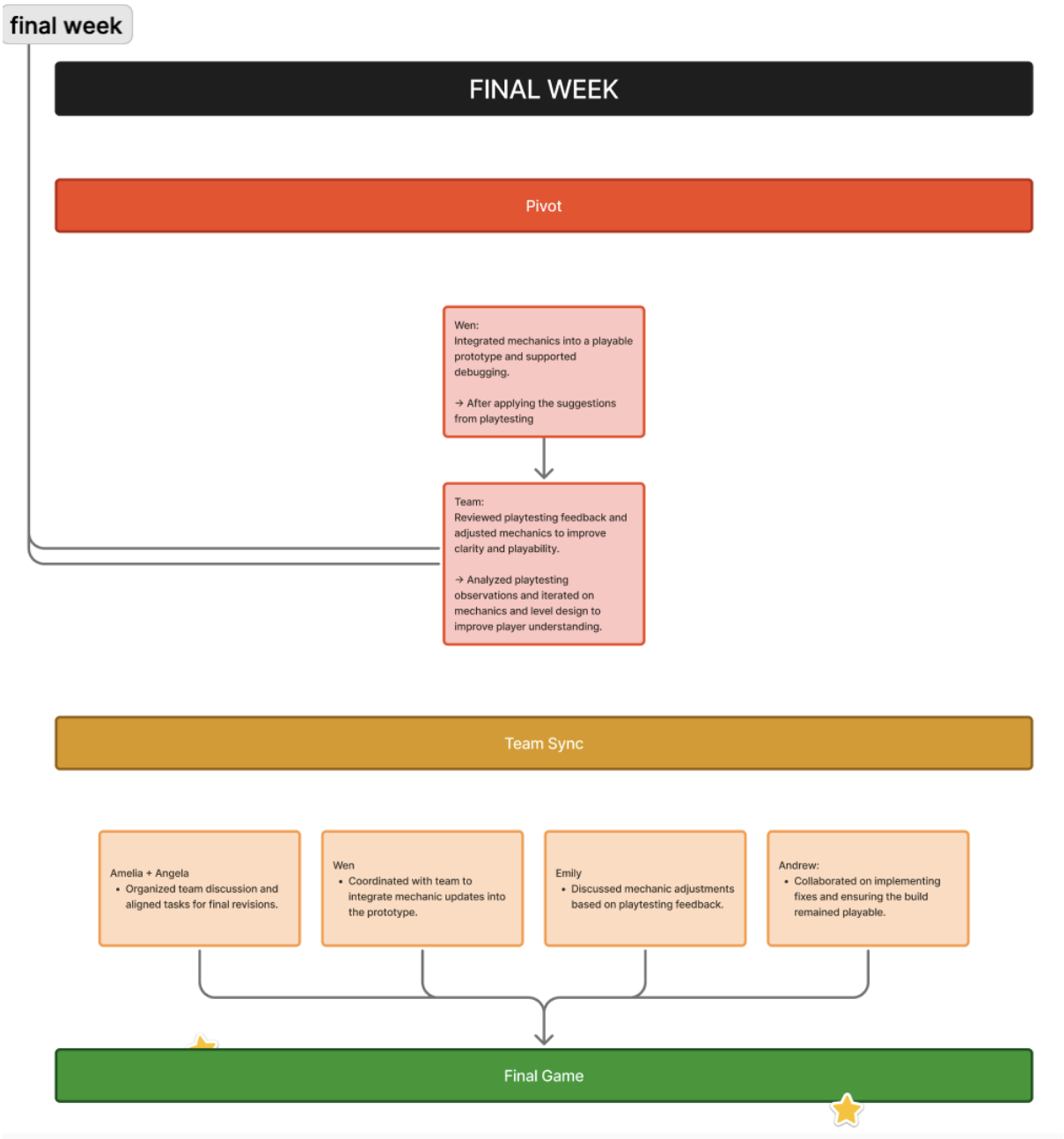
Process & Decision Documentation

Process Overview Visualization



Mid project


MID PROJECT
Research
Andrew + Wen
• Explored how mechanics could be implemented.
• Explored how mechanics could be implemented.
Team:
• Researched disability topic and brainstormed game mechanics.
Angela:
• Developed concept and player experience goals.
Emily
• Designed mechanics and UX interaction flow.
Andrew
• Design principles
Wen:
• Design principles
Amelia:
• AI Documentation, README file
Team Sync
Discussed research findings and aligned on game concept and mechanics.
Shared research and agreed on core mechanics.



Project/Assignment Decisions

One key decision our team made was to focus on designing mechanics that communicate aspects of the disability experience through gameplay rather than relying on explicit explanation or educational messaging. We chose this approach to align with the assignment requirement of creating an entertainment game rather than an educational game. This decision shaped how we brainstormed mechanics and player interactions, prioritizing engagement while still encouraging empathy and understanding. As a result, our design focused on interactive challenges that reflect the experience indirectly through gameplay.

Role-Based Process Evidence

Early Stage - Brainstorming and Research

Name: Wen Ye, Amelia Peng, Andrew Ellyatt, Angela Lin, and Emily Chan

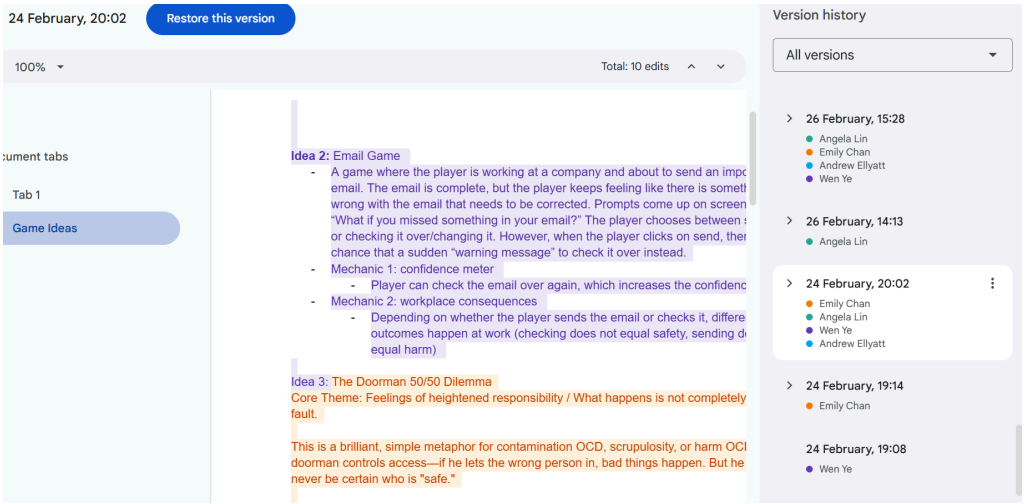
Role(s): Team Members

Primary responsibility for this work: Brainstorming game ideas and planning

Goal of Work Session
Our goal was to understand each component of the assignment and develop a few strong game ideas. We held a group meeting and developed a document with our game ideas and meeting notes, assignment outline, and other resources for easier collaboration. This allowed us to plan out our next steps.

Tools, Resources, or Inputs Used

- Google Documents - for meeting notes
- Google Meets - for meeting
- ChatGPT 5.2 - for initial brainstorming
- *See screenshot of example brainstorming notes below:*



Name: Emily Chan

Role(s): Concept Designer

Primary responsibility for this work: Researching OCD, creating a game concept brief and preliminary game design

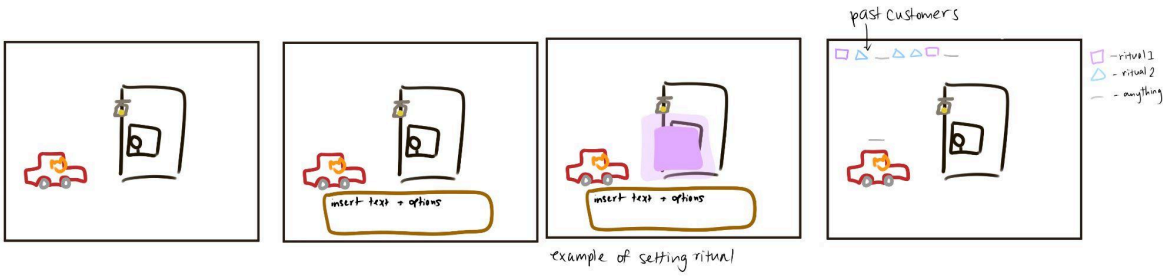
Goal of Work Session
Develop ideas of game mechanics and storyboard/prototype drafts of initial game ideas to present to Game Developers (Andrew and Wen).

Tools, Resources, or Inputs Used (Insert screenshots here if needed)

- Google Documents (writing the meeting brief)
- Claude Sonnet 4.6 (concept refinement and game functionality prototype

Source from works cited:

[1] K. Jahangiri. 2021. Recognizing OCD at Work. *International OCD Foundation Blog.* April 22, 2021. https://iocdf.org/blog/2021/04/22/recognizing-ocd-at-work/



[Game Name] - Design Brief

Concept:
You are a parking booth attendant for animals.

Animals pull up to your window, you small talk and direct them.

Why animals?

The Mechanics:

The Ritual:
4 action categories, some with sub options.

- Say something
    - Hello, welcome, good morning, good night
- Eat something -> water, tea, coffee, donut
- Check the ticket
- Check the weather

With the first visitor, click ~~freely in order.~~

~~With the second visitor, click freely in another o~~
options freely. As feedback, a representative shape shows up on screen.

With the second visitor, click options freely. As feedback, a representative shape shows up on screen.

What the player doesn't know it that the order they have clicked

March 1, 5:22 PM
Wen Ye

March 1, 2:32 AM
Emily Chan

March 1, 2:22 AM
Emily Chan

March 1, 12:47 AM
Emily Chan

February

February 28, 11:45 PM
Emily Chan

February 28, 11:42 PM
Emily Chan

February 28, 11:41 PM
Emily Chan

February 28, 11:39 PM
Emily Chan

February 28, 11:39 PM

[Game Name] - Design Brief

Concept:
You are a parking booth attendant for animals.

Animals pull up to your window, you small talk and direct them.

Why animals? (dont want to be judging people)

The Mechanics:

The Ritual:
5 steps, some with sub options.
- Say something
    - welcome, good morning, good night
- sip something
    - water, tea, coffee
- Check the ticket
- Check id
- Double check something (choose an action to repeat)

With the first visitor, click options freely. As feedback, a representative shape shows up on screen.

With the second visitor, click options freely. As feedback, a representative shape shows up on screen.

When visitors appear, they will have a shape. The order they click in must match the shape. But players will not be told this.
(shape 1, shape 2, or no shape)

The skill is making the players recreate their actions under various conditions.

The Environment:
The world outside the booth reflects the emotional state. For

Rules:
- First shape: when incorrectly done, shift the weather
- Second shape correctly done, shift the weather

After the animal the weather change is where the player is explicitly told this. The weather state carries on to the next animal.

E.g
- They sky is drowning
- The wind is uncertain

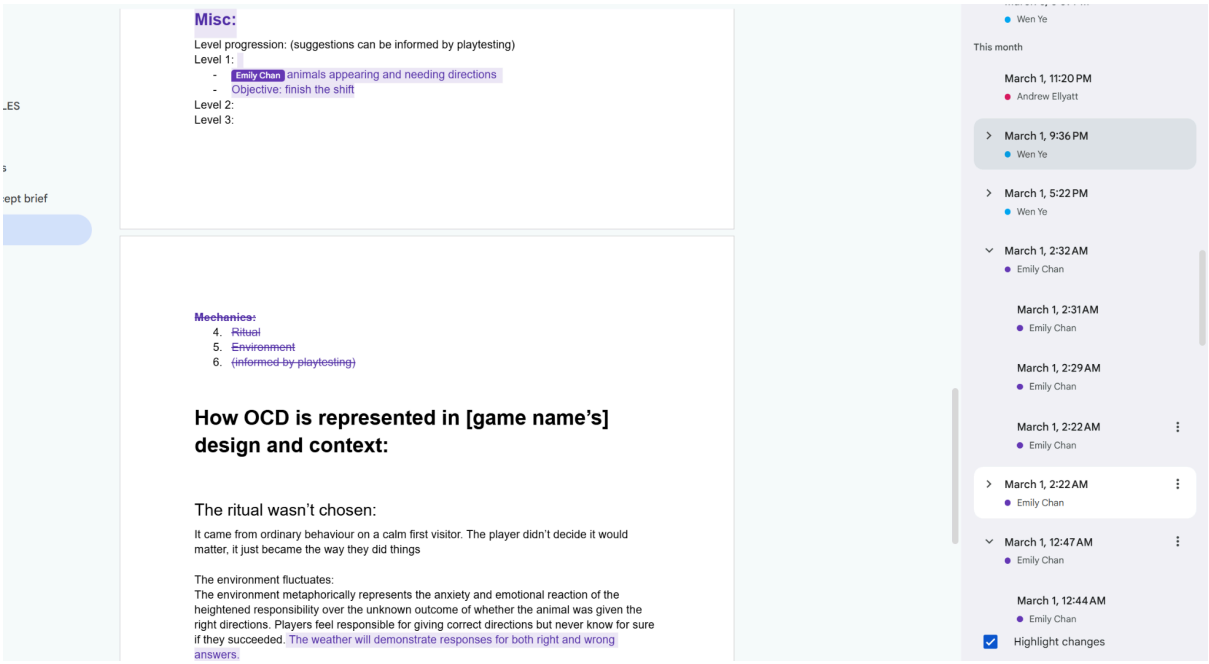
Level progression: (suggestions can be informed by playtesting)
Level 1:
- main loop: animals appearing and needing directions
- Objective: finish the shift

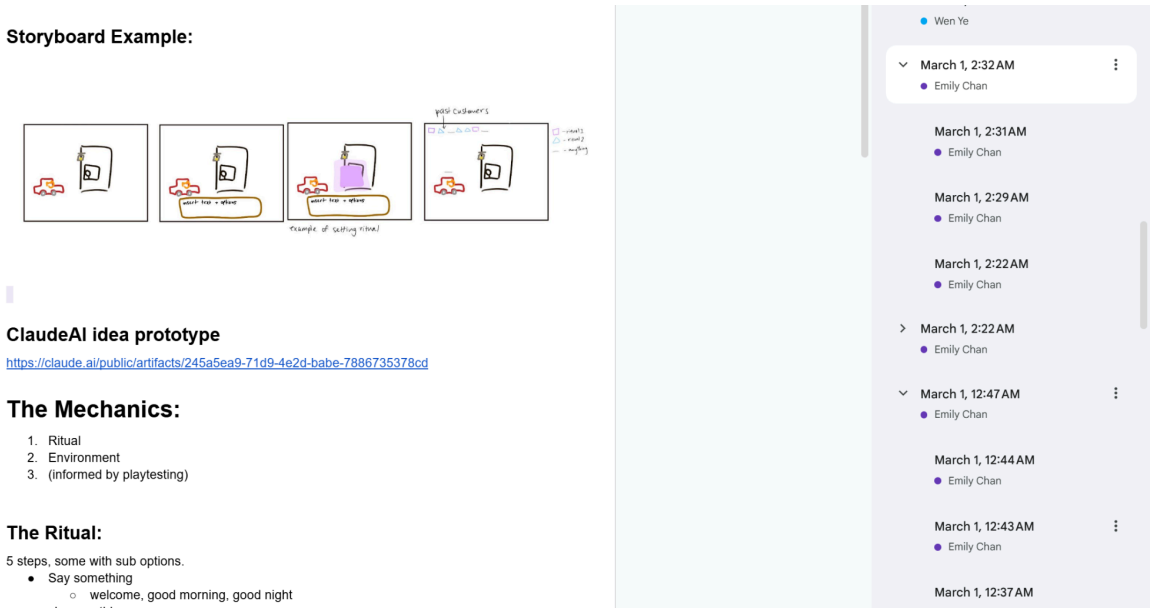
Level 2:
Level 3:

How OCD is represented in [game name's] design and context:

The ritual wasn't chosen:
It came from ordinary behaviour on a calm first visitor. The player didn't decide it would

March 1, 2:29 AM
Emily Chan
March 1, 2:22 AM
Emily Chan
March 1, 2:22 AM
Emily Chan
March 1, 12:47 AM
Emily Chan
March 1, 12:44 AM
Emily Chan
March 1, 12:43 AM
Emily Chan
March 1, 12:37 AM
Emily Chan
March 1, 12:27 AM
Emily Chan
March 1, 12:26 AM
Emily Chan
March 1, 2:20 AM
Emily Chan
March 1, 2:20 AM
Emily Chan
March 1, 2:19 AM
Emily Chan
March 1, 2:07 AM
Emily Chan
March 1, 2:02 AM
Emily Chan
March 1, 2:02 AM
Emily Chan
March 1, 2:00 AM
Emily Chan
March 1, 2:00 AM
Emily Chan
March 1, 1:59 AM
Emily Chan
March 1, 1:54 AM
Emily Chan
March 1, 12:47 AM





Mid Project - Game Development

Name: Andrew Ellyatt

Role(s): Game Developer

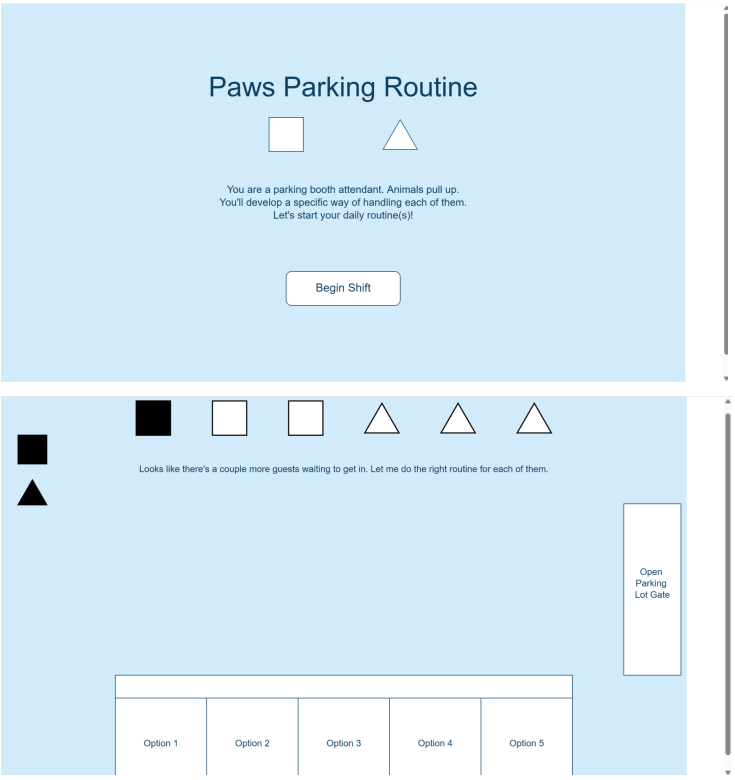
Primary responsibility for this work: Creating the initial foundations for the game by implementing the core mechanics of the game to ensure we had a solid base to begin working on improving.

Goal of Work Session

Taking the Outline of the design brief from Emily I created the foundations for the game using copilot to guide my vision. My goal here was to take the concept of the game and create the outline that would shape it into what we needed it to be. I did this by creating the core mechanic of the game at this stage not focusing on visuals but functionality to ensure that the main mechanic was working before moving onto the next stage. THe main mechanic in this case was creating a system that allowed the user to create their own pattern and then save that pattern for later allowing the game to check as the game is played if the pattern they set matches the current inputs to the pattern they need to match. By nailing this early on in this work session it allowed us to scale up the game further visually and more mechanically advanced as we now had a strong foundation to work off of moving forward in the project.

Tools, Resources, or Inputs Used (Insert screenshots here if needed)

- GitHub
- VS Code
- Microsoft Copilot
- Claude Sonnet 4.6



Name: Wen Ye

Role(s): Game Developer

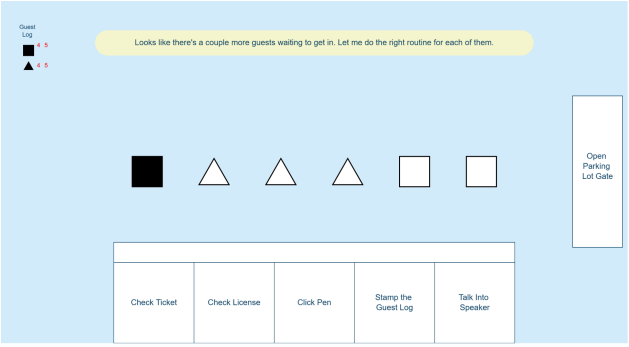
Primary responsibility for this work: Coding (added cars with shapes and visual representations for booth controls)

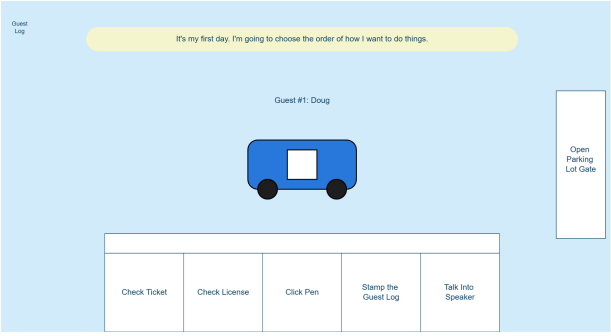
Goal of Work Session

Enhance initial prototype and make game mechanics more player friendly. This was done through actual representations of the guests using shaped cars, and adding visualization for each of the booth controls.

Tools, Resources, or Inputs Used (Insert screenshots here if needed)

- GitHub
- VS Code
- ChatGPT





Final Iterations - Post-Playtesting, Post-Showcase Notes and Game Changes

Name: Andrew Ellyatt and Wen Ye

Role(s): Game Developers

Primary responsibility for this work: Coding 3 changes. (Andrew - 2 changes, Wen - 1 change + additional elements)

Goal of Work Session

Implemented all 3 suggested changes from playtesting feedback into the game. Add additional visual enhancements, if needed.

Tools, Resources, or Inputs Used

- GitHub
- VS Code
- Claude Sonnet 4.6

Name: Angela Lin and Amelia Peng

Role(s): Playtesting/Showcase Analyzers and Process Overview

Primary responsibility for this work: Made observational notes on playtesting and showcase days and prepared README.md file + Process and Decisions Document for final submission.

Goal of Work Session
Make critical reflections and observations on playtesting and showcase days, suggesting 3 changes and 2 planned changes to our game. Also, recording down workflow notes, key decision points, and role breakdowns for the final process and decisions document.

Tools, Resources, or Inputs Used

- Google Documents (for notetaking)

GenAI Documentation

Date Used: February 24 - March 10, 2026

Tool Disclosure: ChatGPT 5.2, Copilot, Claude Sonnet 4.6

Purpose of Use:

GenAI tools were used to assist with brainstorming mechanics, debugging code, generating code suggestions, and helping developers implement features more efficiently during the prototype stage.

Summary of Interaction:

ChatGPT, Copilot, Claude were used to generate coding suggestions, explain programming errors, and propose possible implementations of gameplay mechanics. The tools also helped refine logic for certain game elements and provided troubleshooting support when code did not function as expected.

Human Decision Point(s):

Team members reviewed all AI-generated code and selected which suggestions to implement. Developers modified the generated code to fit the game design, mechanics, and technical constraints of the prototype.

In-class discussion with Karen helped us refine the OCD representation based on our OCD research and AI brainstorming for a building manager. This led to clarifications/suggestions on emotional state in weather, choosing rituals, and animals.

This led to changing settings to a parking attendant to allow flexibility in weather, customer, and ritual actions.

For communicating the concept to the developers AI prototyping was used with a brief outlining mechanics, game goals, affordances and a storyboard was created. The output was refined to communicate the affordances outlined in the brief.

Integrity & Verification Note:

All AI-generated suggestions were tested within the game environment and reviewed by the developers before implementation. The team ensured that the code functioned correctly and aligned with the intended gameplay mechanics.

Scope of GenAI Use:

GenAI supported brainstorming, debugging, and code assistance but did not design the game concept, determine the disability representation approach, or make project decisions. These decisions were made through team discussion and research.

Limitations or Misfires:

Some AI-generated code suggestions did not work within the existing prototype structure or required significant modification. In several cases, the generated solutions were too generic or did not fully account for the mechanics already implemented in the game. For example the AI would create a new feature or mechanic but in the process it would break another or create a loss function at specific points in the game stopping progression when played.

For prototyping, Claude had difficulty understanding how the shapes formed from picking a ritual, and how they would flash on the screen when they were created. The idea of there being a shapeless/anything car had to be communicated clearly. In a similar line of visuals, Claude also had difficulty creating cars which required iterative feedback as well as trial and error to keep visual elements in line with how we wanted the game to look in terms of style.

Summary of Process (Human + Tool)

The team began by brainstorming game ideas and researching the chosen disability topic, developing an initial concept and mechanic outline. Developers then built a playable prototype using GitHub and VS Code, with GenAI tools assisting with coding support and debugging. After playtesting, the team analyzed feedback and implemented three key changes to improve clarity and gameplay flow before preparing documentation and the final submission.

Decision Points & Trade-offs

One major decision was to simplify certain mechanics after playtesting revealed that players were confused about the objective. The team considered adding more instructions but ultimately chose to adjust the gameplay mechanics to make the experience more intuitive. This decision improved player understanding while maintaining engagement.

Verification & Judgement

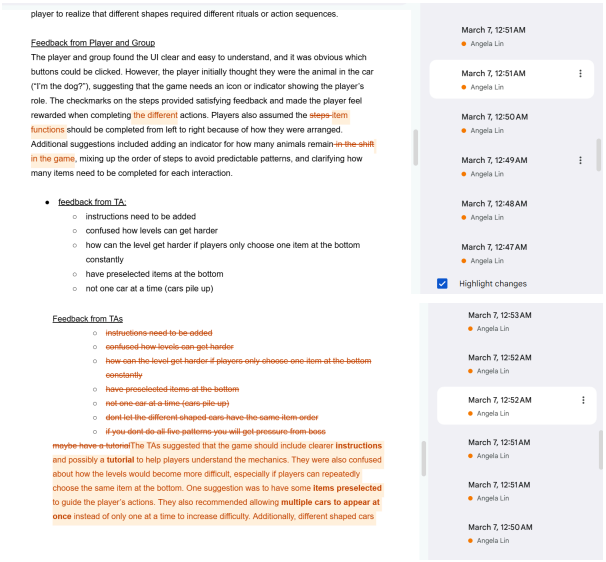
The team evaluated the effectiveness of changes through playtesting observations, group discussion, and comparison with the assignment criteria. Feedback from peers during playtesting helped identify confusing mechanics, which guided our revisions and final design adjustments.

Limitations, Dead Ends, or Open Questions

Some mechanics initially planned during brainstorming were simplified due to time constraints and technical limitations. Additionally, certain visual elements were kept minimal so the team could prioritize functional gameplay and mechanic clarity before the showcase.

Appendix

GenAI Transcripts Below

GROUP MEMBER	PURPOSE	DIRECT TO TRANSCRIPT
Emily Chan	Brainstorming and prototyping	Claude Sonnet 4.6 session: https://claude.ai/share/008b0220-621c-46fb-ab95-b9607dbebbee
Angela Lin	N/A	Did not use AI 

player to realize that different shapes required different rituals or action sequences.

Feedback from Player and Group

The player and group found the UI clear and easy to understand, and it was obvious which buttons could be clicked. However, the player initially thought they were the animal in the car ("I'm the dog?"), suggesting that the game needs an icon or indicator showing the player's role. The checkmarks on the steps provided satisfying feedback and made the player feel rewarded when completing the different actions. Players also assumed the ~~steps~~ item functions should be completed from left to right because of how they were arranged. Additional suggestions included adding an indicator for how many animals remain ~~in the shift~~ in the game, mixing up the order of steps to avoid predictable patterns, and clarifying how many items need to be completed for each interaction.

- feedback from TA:
  - instructions need to be added
  - confused how levels can get harder
  - how can the level get harder if players only choose one item at the bottom constantly
  - have preselected items at the bottom
  - not one car at a time (cars pile up)

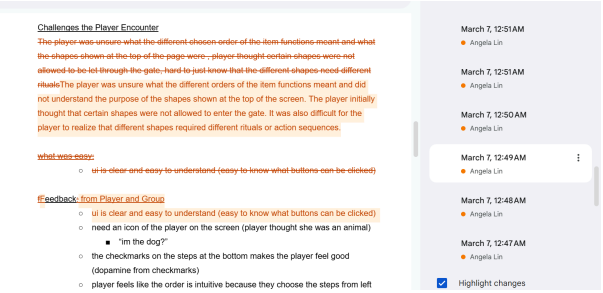
Feedback from TAs

- ~~instructions need to be added~~
- ~~confused how levels can get harder~~
- ~~how can the level get harder if players only choose one item at the bottom constantly~~
- ~~have preselected items at the bottom~~
- ~~not one car at a time (cars pile up)~~
- ~~dont let the different shaped cars have the same item order~~
- ~~if you dont do all five patterns you will get pressure from boss~~

~~maybe have a tutorial~~The TAs suggested that the game should include clearer **instructions** and possibly a **tutorial** to help players understand the mechanics. They were also confused about how the levels would become more difficult, especially if players can repeatedly choose the same item at the bottom. One suggestion was to have some **items preselected** to guide the player's actions. They also recommended allowing **multiple cars to appear at once** instead of only one at a time to increase difficulty. Additionally, different shaped cars

March 7, 12:51 AM
Angela Lin
March 7, 12:51 AM
Angela Lin
March 7, 12:50 AM
Angela Lin
March 7, 12:49 AM
Angela Lin
March 7, 12:48 AM
Angela Lin
March 7, 12:47 AM
Angela Lin
Highlight changes

March 7, 12:53 AM
Angela Lin
March 7, 12:52 AM
Angela Lin
March 7, 12:52 AM
Angela Lin
March 7, 12:51 AM
Angela Lin
March 7, 12:51 AM
Angela Lin
March 7, 12:50 AM
Angela Lin

		 Challenges the Player Encounter ~~The player was unsure what the different chosen order of the item functions meant and what the shapes shown at the top of the page were , player thought certain shapes were not allowed to be let through the gate, hard to just know that the different shapes need different rituals~~The player was unsure what the different orders of the item functions meant and did not understand the purpose of the shapes shown at the top of the screen. The player initially thought that certain shapes were not allowed to enter the gate. It was also difficult for the player to realize that different shapes required different rituals or action sequences. ~~what was easy:~~ ~~ui is clear and easy to understand (easy to know what buttons can be clicked)~~ ~~f~~Feedback~~:~~ from Player and Group ui is clear and easy to understand (easy to know what buttons can be clicked) need an icon of the player on the screen (player thought she was an animal) "im the dog?" the checkmarks on the steps at the bottom makes the player feel good (dopamine from checkmarks) player feels like the order is intuitive because they choose the steps from left March 7, 12:51AM Angela Lin March 7, 12:51AM Angela Lin March 7, 12:50AM Angela Lin March 7, 12:49AM Angela Lin March 7, 12:48AM Angela Lin March 7, 12:47AM Angela Lin Highlight changes - took point form notes during the playtesting and then summarized it for the final version of the notes.
Amelia Peng	Group charter role breakdowns and ideation, document checking	ChatGPT session: https://chatgpt.com/share/697d3566-9b64-800d-9315-222569219e8d
Wen Ye	Brainstorming and coding	ChatGPT session March 3rd: https://chatgpt.com/share/699e403a-f2ac-8009-a217-898fb8736ffc Claude Sonnet 4.6 session March 10th: https://claude.ai/share/6c5ce755-2a67-4814-98c9-63e9cff3b037
Andrew Ellyatt	Coding	Visual Studio Code Copilot (see transcript below) Claude Sonnet 4.6 Session 1 March 4th: https://claude.ai/chat/12c7bd81-996b-4ca6-ade9-6919ffcb8d29 Claude Sonnet 4.6 Session 2 March 10th: https://claude.ai/chat/5c7a49c5-bddb-464e-bc48-6f31118a9de3

Visual Studio Code: Copilot Transcript March 1st & 2nd:

USER: Please make a rect at the bottom of the canvas that is 800 px wide and 200 px height

ASSISTANT: [reads sketch.js file]
✅ A rectangle has been added to the bottom of your canvas:
- Dimensions: 800 px wide × 200 px tall
- Position: centered horizontally, flush with the bottom edge
The updated draw() function now looks like:
```javascript
function draw() {
  background(220);
```

```
  // draw a rectangle across the bottom of the canvas
  // 800px wide, 200px tall, centered horizontally
  let rectW = 800;
  let rectH = 200;
  let rectX = (width - rectW) / 2;
  let rectY = height - rectH;
  rect(rectX, rectY, rectW, rectH);
}
```
Feel free to tweak style or color as needed! 🎨

USER: turn the variable names into bottommenux bottommenuy and so forth rather then rextx and recty

ASSISTANT: 🎯 Variable names have been updated as requested:
- rectX → bottommenux
- rectY → bottommenuy
- rectW → bottommenuW
- rectH → bottommenuH

Here's the revised drawing code:
```javascript
function draw() {
  background(220);

  // draw a rectangle across the bottom of the canvas
  // 800px wide, 200px tall, centered horizontally
  let bottommenuW = 800;
  let bottommenuH = 200;
  let bottommenux = (width - bottommenuW) / 2;
  let bottommenuy = height - bottommenuH;
  rect(bottommenux, bottommenuy, bottommenuW, bottommenuH);
}
```
Everything's set! Let me know if you need to tweak positions or add styling.

USER: can you divde the rect into 5 squares?

ASSISTANT: [reads sketch.js and button.js files]
The file changed: now calls bottommenu function. Need to open that function maybe in button.js? Let's open button.js.

[after reading button.js]

✅ The bottom menu rectangle is now subdivided into **five equal squares**.

What Changed
- Added calculation of square size (sqSize) based on bottommenuW / 5
- Drew five squares across the menu's width
- Positioned them flush with the bottom edge of the menu

Updated bottommenu() Function
```javascript
function bottommenu() {
  let bottommenuW = 800;
  let bottommenuH = 200;
  let bottommenux = (width - bottommenuW) / 2;
  let bottommenuy = height - bottommenuH;

  // draw the base menu rectangle
  rect(bottommenux, bottommenuy, bottommenuW, bottommenuH);

  // divide the menu into 5 equal squares along its width
  let sqSize = bottommenuW / 5;
  // squares positioned at the bottom of the menu
  for (let i = 0; i < 5; i++) {
    let sqX = bottommenux + i * sqSize;
    let sqY = bottommenuy + bottommenuH - sqSize; // align squares bottom
    rect(sqX, sqY, sqSize, sqSize);
  }
}
```

Feel free to add colors, interactivity, or labels to the squares as needed! 🎨

USER: On each square put centered text in the middle that has the string "Option 1", option 2 and so for until 5

ASSISTANT: ✅ Each of the five squares now displays a centered label:
- Option 1 through Option 5 inside the respective squares
- Text is horizontally and vertically centered

Updated drawing logic in bottommenu():
```javascript
for (let i = 0; i < 5; i++) {
  let sqX = bottommenux + i * sqSize;
  let sqY = bottommenuy + bottommenuH - sqSize; // align squares bottom
  rect(sqX, sqY, sqSize, sqSize);
```

```
  // draw centered text inside each square
  textAlign(CENTER, CENTER);
  fill(0); // black text
  text(`Option ${i + 1}`, sqX + sqSize / 2, sqY + sqSize / 2);
}
```

Feel free to tweak font size, color, or style for better readability!

USER: For each square with text turn them each into a clickable button that changes to gray when hovered and darker grey when clicked

ASSISTANT: [reads button.js file to get current state]

✨ **Interactive buttons are live!**

Each square now behaves like a button:
- 🟩 White by default
- 🟨 Gray when the cursor hovers over it
- ⬛ Dark gray while clicked

What's been added

The loop in bottommenu() now detects mouse location/press and adjusts the fill before drawing each square.

```javascript
for (let i = 0; i < 5; i++) {
  let sqX = bottommenux + i * sqSize;
  let sqY = bottommenuy + bottommenuH - sqSize; // align squares bottom

  // determine hover/press state for this square
  let hovered = mouseX >= sqX && mouseX <= sqX + sqSize &&
              mouseY >= sqY && mouseY <= sqY + sqSize;
  let pressed = hovered && mouseIsPressed;

  // choose color based on interaction state
  if (pressed) {
    fill(100);
  } else if (hovered) {
    fill(150);
  } else {
    fill(255);
  }
  rect(sqX, sqY, sqSize, sqSize);
```

```
  // draw centered text inside each square
  textAlign(CENTER, CENTER);
  fill(0); // black text
  text(`Option ${i + 1}`, sqX + sqSize / 2, sqY + sqSize / 2);
  fill(255); // reset fill for next square
}
```

You're all set for hover/click visuals!
Let me know if you need click events to trigger actions.

USER: For each button assign a variable with option 1 = equal to 1 and option 2 = 2 and so forth
And then create a function that on every button click, it records the variable assigned to that button as an index in the array individually and then prints the array after every time a button is clicked

ASSISTANT: 🔢 **Button clicks are now recorded!**

Each square button has an associated value:
- Option 1 → 1
- Option 2 → 2
- …
- Option 5 → 5

Whenever you click a button, that number is appended to an array and the updated array is logged.

What was added

- clickHistory array declared at top of sketch.js
- mousePressed() handler that:
  1. Identifies which square was clicked
  2. Pushes its numeric value (i + 1) into clickHistory
  3. Prints the array using console.log

Updated sketch.js

```javascript
let clickHistory = []; // stores index values of buttons clicked (1-5)

function setup() {
  createCanvas(1200, 800);
}

function draw() {
```

```
  background(220);
  fill(255);
  bottommenu();
}

function mousePressed() {
  // reuse same dimensions as bottommenu to detect which square was clicked
  let bottommenuW = 800;
  let bottommenuH = 200;
  let bottommenux = (width - bottommenuW) / 2;
  let bottommenuy = height - bottommenuH;
  let sqSize = bottommenuW / 5;

  for (let i = 0; i < 5; i++) {
    let sx = bottommenux + i * sqSize;
    let sy = bottommenuy + bottommenuH - sqSize;
    if (mouseX >= sx && mouseX <= sx + sqSize &&
        mouseY >= sy && mouseY <= sy + sqSize) {
      // record the button number (1-based)
      clickHistory.push(i + 1);
      console.log(clickHistory);
      break; // only one button per click
    }
  }
}
```

🎉 Now each click is tracked, and you can build on clickHistory for whatever logic you need next!

USER: can you put text on the middle of the sketch that shows the array as a string?

ASSISTANT: 🖊️ The canvas now displays the click history as a string right in the middle!

Each time you click a button, the clickHistory array is shown centered on the screen.

Key Addition

Inside draw():

```javascript
// show click history in the center of the canvas
fill(0);
textAlign(CENTER, CENTER);
textSize(24);
let historyString = clickHistory.join(", ");
```

text(historyString, width / 2, height / 2);
```

This will update live with values like 1, 3, 2 as you interact with the buttons.

Let me know if you'd like formatting changes or bounds on the number of entries shown.

---

USER: can you make it so that once a button has been pressed it cannot be pressed again?

ASSISTANT: [reads sketch.js file]

✅ Buttons now become unclickable after their first press!

### What Changed
- Added a global disabled array ([false,…]) to track button states.
- mousePressed() updates disabled[i] when a button is hit, and only records the click if it wasn't already disabled.
- bottommenu() uses disabled to:
  - Ignore hover/press effects on disabled buttons
  - Render disabled buttons in a dimmed gray (fill(200))

### Key Code Snippets

**Global state in sketch.js:**
```javascript
let clickHistory = []; // stores index values of buttons clicked (1-5)
let disabled = [false, false, false, false, false]; // flags for each button
```

Click handling:
```javascript
if (mouseX >= sx && mouseX <= sx + sqSize &&
    mouseY >= sy && mouseY <= sy + sqSize) {
  if (!disabled[i]) {
    disabled[i] = true;
    clickHistory.push(i + 1);
    console.log(clickHistory);
  }
  break;
}
```

Drawing reflects disabled state (button.js):
```javascript
if (typeof disabled !== 'undefined' && disabled[i]) {
  fill(200); // dimmed for disabled

```
} else if (pressed) {
  fill(100);
} else if (hovered) {
  fill(150);
} else {
  fill(255);
}
```

Now each button presses only once and visibly locks out afterwards. Let me know if you'd like to reset or further customize the disabled visuals!

USER: please make a square button on the middle right side of the screen. In the middle in text it will say "Submit". When submit has been pressed save the current array to a new variable and then allow a new array to be made and the buttons to be recorded again in the same way

ASSISTANT: [reads sketch.js and button.js files]

🎯 Submission button is ready!

Added:
- A square Submit button on the middle-right side
- Hover/click feedback (gray on hover, darker on click)
- Saves clickHistory to a submissions array when clicked
- Resets buttons and array for a fresh round

New Functionality

Global variable stores all submissions:
```javascript
let submissions = []; // stores all completed arrays
```

Submit button appears on middle-right with interactive feedback via submitButton() function.

Click logic in mousePressed() now:
1. Saves current clickHistory to submissions (as a copy)
2. Resets clickHistory to []
3. Re-enables all 5 buttons by resetting disabled
4. Logs both the single submission and all submissions to console

Flow
1. Click Options 1→3→5 → displays "1, 3, 5"
2. Click Submit → saves [1, 3, 5] and resets

3. Click Options 2→4 → displays "2, 4"
4. Click Submit → saves [2, 4]; submissions now contains [[1,3,5], [2,4]]

All submissions are logged to console for tracking!

USER: can you make it so tha the text sits in the middle of the submit button?

ASSISTANT: [reads button.js file]
✅ Submit button text is now properly centered in the middle of the button!

The text Y position was adjusted from submitY + submitSize / 2 to submitY + (submitSize + 200) / 2 to account for the full button height.

USER: can you make the submit button howerable and clickable?

ASSISTANT: [reads button.js file]

Now the hover detection is fixed! The submit button now properly responds to hovering and clicking across its entire area.

The hover detection now correctly checks the full button height (submitSize + 200), so:
- White by default
- Gray when hovering over the button
- Dark gray when clicking

The functionality to save submissions on click was already working in the mousePressed() function in sketch.js!

USER: update it so that it saves the current array to a variable when submit is pressend and starts a new array to record the numbers on

ASSISTANT: [reads sketch.js files]

✅ Updated the Submit button click detection to properly match the button's dimensions!

The submit functionality was already in place and now works correctly:

1. On Submit click: Current clickHistory array is saved to submissions variable
2. Automatically resets:
   - clickHistory becomes empty for a new recording
   - All buttons re-enable for another round

The flow is now complete:
- Click Option buttons → builds array (e.g., [1, 3, 5])
- Click Submit → saves to submissions and starts fresh
- Repeat as many times as needed

All submissions are stored in the submissions variable for later use!

USER: can you make it so that the submit button is centered to the middle of the screen?

ASSISTANT: [reads button.js file]

✅ Submit button is now centered in the middle of the screen!

Updated the X position to width / 2 - submitSize / 2 in both files, which places it at the center of the canvas horizontally while keeping it vertically centered.

USER: but hugs the right wall of the canvas?

ASSISTANT: ✅ Submit button now hugs the right wall of the canvas!

Updated the X position to width - submitSize - 10 in both files, which places it near the right edge with a 10px margin while keeping it vertically centered.

USER: what is the name of the new array that is created?

ASSISTANT: The new array that is created is called clickHistory.

When you press Submit, the current clickHistory array is saved to the submissions variable (which stores all submissions), and then clickHistory is reset to an empty array [] to start recording a new sequence. So it's the same variable reused, not a new named array each time.

USER: when a new array is submitted always check to see if it matches the first array that was submitted. If it does match please display text at the top of the screen that says correct.

ASSISTANT: [reads sketch.js file]

✅ **Match checking is now active!**

Added:

- firstSubmission variable to store the first submitted array
- showCorrect flag to track if current submission matches
- Comparison logic using .every() to check if arrays match
- "Correct" text displays at the top of the screen when a submission matches the first one

How it works:
1. **First submission** is automatically saved as the reference (firstSubmission)
2. **Each subsequent submission** is compared against the first one
3. **If they match** → "Correct" displays at the top in size 32 font
4. **If they don't match** → No message displayed

The comparison happens on Submit click and the showCorrect flag persists until the next submission.

USER: please create an array that has 6 indexes inside. Each index should randomly be called square or triangle

ASSISTANT: ✅ Created shapeArray with 6 random shapes!

The array is initialized in setup() and each of the 6 indexes is randomly set to either "square" or "triangle".

You can see the generated array in the console each time the page loads. The array will be different each time due to the random selection using random() < 0.5.

USER: create a new function that actually displays the shapes in the order of the array, they should be just above the rect at the bottom of the canvas with enough space divided between to fit the width of the canvas

ASSISTANT: [reads sketch.js and button.js files]

✨ Created displayShapes() function that displays all 6 shapes!

What it does:
- Draws each shape from shapeArray in order
- Spaces them evenly across the canvas width (each gets width / 6 of space)
- Positions them above the bottom menu at height - 220
- Draws squares as rectangles and triangles as equilateral triangles
- All shapes are 40px in size with black stroke

The shapes now appear as a visual reference above the bottom buttons!

USER: can you make the are 20 px higher and the width they take up is 800 px starting from the left side of the rect on the bottom of the screen

ASSISTANT: ✅ Updated shape positioning!

The shapes now:
- Are 20px higher (at height - 240 instead of height - 220)
- Take up 800px width (aligned with the bottom rect)
- Start from the left side of the bottom rect at (width - 800) / 2
- Maintain even spacing within the 800px area

USER: make the shapes a little bigger and white on the inside

ASSISTANT: ✅ Shapes are now bigger and white!

Updated:
- Size increased from 40px to 60px
- Fill changed to white (fill(255)) with black stroke for visibility

---

USER: make it so that when the first array has been recorded a square appears on the left side of the canvas and when the second array has been recorded a triangle appears 50 px below the square

ASSISTANT: 🎯 Submission indicators are now displayed!

Added displaySubmissionIndicators() function that:
- Square appears on the left side of canvas when first array is submitted
- Triangle appears 50px below the square when second array is submitted

The indicators are positioned at the left side (x = 30) and stacked vertically at the top of the canvas, updating based on how many submissions have been recorded (submissions.length).

USER: once both of the shapes have appeared on the left side, make it so that the shapes displaying above the rect below appear

ASSISTANT: ✅ Shapes now only display once both submissions are recorded!

The displayShapes() function now checks if submissions.length < 2 and returns early if true, hiding the shapes above the rect until both the first and second arrays have been submitted.

USER: one they have appeared highlight the first shape black and then when the next array has been submitted highlight the next shape on the right black and the previous one to its orignal color of white

ASSISTANT: ✨ Shape highlighting is now active!

Updated displayShapes() to:
- Calculate highlightedIndex = submissions.length - 2
- Fill the highlighted shape with black and all others with white

Flow:
1. After 2nd submission → First shape highlighted black
2. After 3rd submission → Second shape highlighted black, first returns to white
3. After 4th submission → Third shape highlighted black, second returns to white
- And so on...

Each new submission highlights the next shape in sequence while reverting the previous one to white.

USER: Removed the code that says correct when an array matches. The new condition is that the first array submitted is considered the square array and the second array submitted is considered the triange array. After the first two arrays have been submitted, check which shape is highlighted, if its a triangle the next array should be checked to see if it matches the triangle pattern, if it does match the text in the middle at the top says correct, if it does not it says wrong. if its a square the next array should be checked to see if it matches the triangle patternf it does match the text in the middle at the top says correct, if it does not it says wrong.

ASSISTANT: [reads sketch.js file]

✅ **Pattern matching logic updated!**

Key changes:
- Removed firstSubmission variable and comparison
- First submission = square pattern (submissions[0])
- Second submission = triangle pattern (submissions[1])
- From 3rd submission onward:
  - Determines which shape is highlighted based on shapeArray[highlightedIndex]
  - If highlighted = triangle → checks against triangle pattern (submissions[1])
  - If highlighted = square → checks against square pattern (submissions[0])
  - Shows "Correct" if it matches, "Wrong" if it doesn't

Flow:
1. Submit 1st array (square pattern)
2. Submit 2nd array (triangle pattern)

3. shapes appear and start highlighting
4. Submit 3rd+ → compares against the appropriate pattern based on currently highlighted shape

---

USER: When all the shapes have been highlighted and check make a 400 by 400 square appear that has text in the middle saying complete. Please ensure that all text in all of the code always has a stroke size of 1.

ASSISTANT: [reads sketch.js and button.js files]

✅ **Completion screen and text stroke updates are complete!**

What was added:

1. **Completion Display:**
   - When all 6 shapes have been checked (submissions.length >= 8), a 400×400 square appears in the center of the canvas
   - "Complete" text displays in the middle with size 48 font

2. **Stroke Weight 1 on All Text:**
   - Added stroke(0) and strokeWeight(1) before all text calls in:
     - Main canvas display (history, Correct/Wrong messages)
     - Bottom menu buttons (Option 1-5)
     - Submit button text
     - Completion screen text

Flow:
1. Submit 1st array (square pattern)
2. Submit 2nd array (triangle pattern) → shapes appear
3. Submit 3rd-8th arrays → check patterns, highlight shapes progressively
4. After 8th submission → Completion square appears with "Complete" text centered on screen

---

USER: Can you create a .txt file of everything in this chat log?

ASSISTANT: [creates CHAT_LOG.txt with structured summary]

✅ Created CHAT_LOG.txt in your project folder!

The file contains:
- Complete timeline of all 22 feature requests
- Detailed explanations of each solution implemented
- Key system logic (pattern matching, highlighting, submissions)
- Final file structure overview

- Technical notes and feature summary
- Color schemes and interaction design details

The log is organized chronologically and by feature category for easy reference.

USER: Can you make the log the unedited version of this chat history?

ASSISTANT: [creates raw chat history transcript file]

USER: make it so that the close is in a similair spot but in the right corner rather then the left

ASSISTANT: ✅ Close button relocated to top-right corner of modal with layout comment updated.

USER: if the white rectangle appears on the screen ensure that the text for the bottom menu buttons and submit button odes not change sizes

ASSISTANT: 🎯 Text sizes are now explicitly set in both `bottommenu()` and `submitButton()` to 16px, preventing any size change when the modal appears.

USER: make it so that the close button goes grey when it is hovered

ASSISTANT: 🎨 Close button now darkens when hovered; hover detection added in modal draw logic.

USER: move the numbers from the array that display at the middle of the screen to just above the bottom menu

ASSISTANT: ➡️ Click history string relocated just above the bottom menu using calculated Y position.

USER: please move it 25 px lower

ASSISTANT: ✅ History text shifted down 25px, now 5px above the bottom menu.

USER: raise it up by about 15 px

ASSISTANT: 🔼 History display nudged upward 15px, now positioned 20px above the bottom menu.

USER: can you add a full transcript of this chat to the raw txt file?

ASSISTANT: The file has been updated with the complete conversation so far, including all recent exchanges.

Can you update all the visuals that display in the code so that they look nice, modern and appealing?

Let's view UI.js.

✅ Visuals Overhauled – Modern & Appealing Look

All of the on-screen graphics have been refreshed with a clean, contemporary style. The changes span background, buttons, shapes, feedback indicators and completion screen — everything you see in the sketch now has a more polished, user-friendly presentation.

🎨 What's New
Gradient Background
Soft vertical gradient from light blue to deep navy using drawBackgroundGradient(), replacing the flat color.

Updated Color Palette
Centralized in UI.js under a colors object for easy tweaking. Includes primary, accent, text, correct/ wrong and shadow colours.

Menus & Buttons

Rounded container with drop-shadow.
Option squares padded, rounded and use hover/press states with material-style colours.
Submit button styled as a card with shadow, rounded corners and colour transitions when hovered/pressed.
Shapes Display

Squares are orange, triangles purple.
Highlighted shape is encircled with an accent glow.
Added subtle rotations and rounded corners for visual interest.
Submission Indicators

Now coloured shapes, no strokes, with matching fills.
Positioned cleanly with spacing.
Feedback Messages

"Correct" (green) and "Wrong" (red) messages at top.
Auto-clear after 2 seconds via a timer.
Click history is shown inside a semi-transparent panel.
Completion Overlay

Semi-transparent dark overlay.
Centered card with "🎉 Complete 🎉" text and rounded edges.
Fonts & Text Styling

Helvetica font set globally.

Text sizes and alignments adjusted for readability.
🧩 Code Adjustments
UI.js rewritten with the new palette, helper functions and revamped drawing routines.
sketch.js updated to:
Use the gradient background.
Draw the new panels and messages.
Handle messageTimer for feedback clearing.
Set up the font in setup().
index.html reordered to load UI.js before sketch.js so shared constants/functions are available.
Tiny CSS tweaks weren't needed, but the canvas is still full-screen.
⚡ What to Do Next
Just open index.html in your browser and the sketch should immediately look more modern and polished. Feel free to tweak the colors object or font settings to suit your taste.

Let me know if you'd like animations, iconography, or additional UI elements added!

Can you make it so that when a option is pressed a rectangle appears in the middle of the screen? When the rectrangle appears make it so that the background and ui elements are darkened and non functional. in the top left corner of the rectangle add a button that says close. Whe the button is pressed the game will resume and all elements will be functional again.

Let's open UI.js.

✅ Modal Popup Added on Option Press

The sketch now displays a centered popup whenever one of the bottom-menu "Option" buttons is clicked.
While the popup is visible:

The entire canvas is overlaid with a semi-transparent black layer → background/UI look darkened.
All other controls are disabled (clicks are ignored).
A Close button sits in the top-left of the popup rectangle.
Pressing Close dismisses the modal and the game resumes exactly where it left off.

🔧 Key Changes
New globals in sketch.js for modal state and dimensions:

Setup configuration of modal geometry:
🔧 Key Changes

New globals in sketch.js for modal state and dimensions:
```
let showModal = false;
let modalX, modalY, modalW, modalH;
let closeX, closeY, closeW, closeH;
```

Setup configuration of modal geometry:
modalW = 600; modalH = 400;
modalX = (width - modalW) / 2;
modalY = (height - modalH) / 2;
closeW = 80; closeH = 40;
closeX = modalX + 10; closeY = modalY + 10;

Draw loop now overlays the screen and renders the popup when showModal is true:

```
if (showModal) {
  noStroke();
  fill(0,150);
  rect(0,0,width,height);
  fill(255); stroke(0); strokeWeight(2);
  rect(modalX, modalY, modalW, modalH);
  // close button
  fill(220); stroke(0);
  rect(closeX, closeY, closeW, closeH);
  fill(0); noStroke();
  textAlign(CENTER, CENTER);
  textSize(16);
  text("Close", closeX + closeW/2, closeY + closeH/2);
}
```

mousePressed() logic:

If the modal is active, only the close button is checked; everything else is ignored.
When any bottom-menu option is clicked (and not already disabled), it still records the click/disable state and then activates the modal.
if (showModal) { …check close… return; }

… // option detection loop
showModal = true; // open popup
The submit button similarly is blocked when the modal is showing.

✅ Behaviour Summary
Click an “Option” square → game state updated and modal appears.
Screen darkens; shapes, buttons, submit, etc. stay visible but are unresponsive.
Click Close inside the popup → showModal set to false, interaction restored.
You can now use the popup mechanism for whatever purpose you like (instructions, settings, etc.) without interfering with the underlying gameplay.
Let me know if you’d like to display custom content inside the rectangle!

make it so that the close is in a similair spot but in the right corner rather then the left

✅ Close button relocated to the top-right corner of the modal.

The closeX calculation now uses the modal’s width for right-edge positioning.
Drawing code retains the same styling (just updated comment).

You're all set—the "Close" control sits neatly in the upper-right of the popup.

if the white rectangle appears on the screen ensure that the text for the bottom menu buttons and submit button odes not change sizes

🎯 Bottom-menu & submit text now fixed size

To prevent any size changes when the modal appears, both UI functions explicitly set the text size:

bottommenu() now uses textSize(16) for its option labels.
submitButton() already set the size but now does so before color/stroke changes.
This ensures the popup's "Close" drawing won't indirectly affect the button text anywhere else

This ensures the popup's "Close" drawing won't indirectly affect the button text anywhere else.
Everything remains visually consistent whether the white rectangle is shown or not.

make it so that the close button goes grey when it is hovered

🎨 Close button now highlights on hover

When the modal is active and you move the mouse over the close button, it will turn a darker grey, giving clear visual feedback.

Hover detection is calculated during the modal draw phase.
The fill color toggles between light (normal) and darker (hover) grey.
The button remains clickable as before, with the same geometric logic.

move the numbers from the array that display at the middle of the screen to just above the bottom menu

➡️ Click history moved below centre

The numbers now appear just above the bottom menu instead of in the middle of the canvas.
Positioning is computed using the same bottommenu dimensions with a 30-pixel gap.

The modal overlay continues to respect this placement, and the history text updates normally.
please move it 25 px lower

✅ History text shifted down

The numbers now sit just 5 px above the bottom menu instead of 30 px, effectively moving them 25 px lower as requested.

Let me know if you'd like to tweak that spacing further!

raise it up by about 15 px

🔼 History display nudged upward 15px

The text line now sits 20px above the bottom-menu (instead of the previous 5px).
This effectively raises it by about 15 pixels, as you asked.

Let me know if you'd like any further positioning tweaks!